



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 02

Lab Report Name : Shortest Job First (SJF) Non-Preemptive Scheduling Algorithm.

Date of Submission : 09.08.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

An application is to be developed to implement the **Shortest Job First–Non-Preemptive (SJF-NP) CPU Scheduling Algorithm**. In this scheduling technique, the process with the **smallest burst time** among all processes that have already arrived and are ready for execution is selected next. Since the algorithm is non-preemptive, once a process starts execution, it continues until its completion without interruption.

The processes may have **different arrival times**, and a process can be scheduled only after it has arrived in the ready queue. The application must determine the scheduling order and calculate the required performance measures, including **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** for each process.

2. Objectives:

The objectives of this lab experiment are:

- To understand the operational principles of the **Shortest Job First (SJF) CPU scheduling algorithm**.
- To implement the **non-preemptive SJF scheduling algorithm** based on the burst time of processes.
- To handle processes with **different arrival times** and manage CPU idle time when no process is ready.
- To calculate and analyze the scheduling performance metrics, including **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)**.
- To evaluate the efficiency of SJF scheduling and understand its performance in comparison with other CPU scheduling algorithms.

3. Souce Code Screenshot:

```
#include <bits/stdc++.h>
using namespace std;

struct Process {
    string pid;
    int arrival_time;
    int burst_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
    bool completed;
};

int main() {
    int num_processes;
    cout << "Enter the number of processes: ";
    cin >> num_processes;

    vector<Process> processes(num_processes);

    cout << "Enter PID, Arrival Time, Burst Time for each process:\n";
    for (int i = 0; i < num_processes; i++) {
        cin >> processes[i].pid >> processes[i].arrival_time >> processes[i].burst_time;
        processes[i].completed = false;
    }

    int current_time = 0;
    int completed_processes = 0;

    while (completed_processes < num_processes) {
        int shortest_job = -1;
        int min_burst_time = INT_MAX;
```

```
        if (!processes[i].completed &&
            processes[i].arrival_time <= current_time &&
            processes[i].burst_time < min_burst_time) {
            min_burst_time = processes[i].burst_time;
            shortest_job = i;
        }
    }

    if (shortest_job == -1) {
        int next_arrival = INT_MAX;
        for (int i = 0; i < num_processes; i++) {
            if (!processes[i].completed && processes[i].arrival_time > current_time) {
                next_arrival = min(next_arrival, processes[i].arrival_time);
            }
        }
        current_time = next_arrival;
    } else {
        current_time += processes[shortest_job].burst_time;
        processes[shortest_job].completion_time = current_time;
        processes[shortest_job].turnaround_time = processes[shortest_job].completion_time - processes[shortest_job].arrival_time;
        processes[shortest_job].waiting_time = processes[shortest_job].turnaround_time - processes[shortest_job].burst_time;
        processes[shortest_job].completed = true;
        completed_processes++;
    }
}

cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
double total_waiting = 0, total_turnaround = 0;
```

```

cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
double total_waiting = 0, total_turnaround = 0;

for (const auto &p : processes) {
    cout << p.pid << "\t" << p.arrival_time << "\t" << p.burst_time << "\t"
        << p.completion_time << "\t" << p.turnaround_time << "\t" << p.waiting_time << endl;
    total_waiting += p.waiting_time;
    total_turnaround += p.turnaround_time;
}

cout << fixed << setprecision(2);
cout << "\nAverage Waiting Time: " << total_waiting / num_processes << endl;
cout << "Average Turnaround Time: " << total_turnaround / num_processes << endl;

return 0;
}

```

Figure 1: Source Code of the Comparison of SJF Algorithm

```

#include <bits/stdc++.h>
using namespace std;
struct Process {
    string pid;
    int arrival_time;
    int burst_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
};

double FCFS(vector<Process> p, double &avg_tat) {
    int n = p.size();
    int current_time = 0;

    double total_waiting = 0;
    double total_turnaround = 0;
    sort(p.begin(), p.end(), [](Process a, Process b) {
        return a.arrival_time < b.arrival_time;
    });

    for (int i = 0; i < n; i++) {
        if (current_time < p[i].arrival_time)
            current_time = p[i].arrival_time;

        current_time += p[i].burst_time;

        p[i].completion_time = current_time;

        p[i].turnaround_time =
            p[i].completion_time - p[i].arrival_time;

        p[i].waiting_time =
            p[i].turnaround_time - p[i].burst_time;

        total_waiting += p[i].waiting_time;
        total_turnaround += p[i].turnaround_time;
    }

    cout << "\n===== FCFS =====\n";

    cout << "PID\tAT\tBT\tCT\tTAT\tWT\n";

    for (auto &x : p) {
        cout << x.pid << "\t"
            << x.arrival_time << "\t"
            << x.burst_time << "\t"
            << x.completion_time << "\t"
            << x.turnaround_time << "\t"
            << x.waiting_time << endl;
    }

    double avg_wt = total_waiting / n;
    avg_tat = total_turnaround / n;

    cout << fixed << setprecision(2);

    cout << "\nAverage Waiting Time: "
        << avg_wt << endl;

    cout << "Average Turnaround Time: "
        << avg_tat << endl;

    return avg_wt;
}

double SJF(vector<Process> p, double &avg_tat) {
    int n = p.size();

    vector<bool> completed(n, false);

    int current_time = 0;
    int completed_processes = 0;

    double total_waiting = 0;
    double total_turnaround = 0;

    while (completed_processes < n) {
        int shortest_job = -1;
        int min_burst_time = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (!completed[i] &&
                p[i].arrival_time <= current_time &&
                p[i].burst_time < min_burst_time) {
                min_burst_time = p[i].burst_time;
                shortest_job = i;
            }
        }

        if (shortest_job == -1) {

```

```

if (shortest_job == -1) {
    int next_arrival = INT_MAX;

    for (int i = 0; i < n; i++) {
        if (!completed[i] &&
            p[i].arrival_time > current_time) {
            next_arrival =
                min(next_arrival, p[i].arrival_time);
        }
    }

    current_time = next_arrival;
}

else {
    current_time += p[shortest_job].burst_time;

    p[shortest_job].completion_time = current_time;

    p[shortest_job].turnaround_time =
        p[shortest_job].completion_time -
        p[shortest_job].arrival_time;

    p[shortest_job].waiting_time =
        p[shortest_job].turnaround_time -
        p[shortest_job].burst_time;

    while (completed_processes < n) {
        else {
            total_waiting += p[shortest_job].waiting_time;
            total_turnaround += p[shortest_job].turnaround_time;

            completed[shortest_job] = true;
            completed_processes++;
        }
    }

    cout << "\n===== SJF =====\n";

    cout << "PID\tAT\tBT\tCT\tTAT\tWT\n";

    for (auto &x : p) {
        cout << x.pid << "\t"
            << x.arrival_time << "\t"
            << x.burst_time << "\t"
            << x.completion_time << "\t"
            << x.turnaround_time << "\t"
            << x.waiting_time << endl;
    }

    double avg_wt = total_waiting / n;
    avg_tat = total_turnaround / n;

    cout << fixed << setprecision(2);

    cout << "\nAverage Waiting Time: "
        << avg_wt << endl;

    cout << "Average Turnaround Time: "

```

```

    return avg_wt;
}

int main() {
    int num_processes;

    cout << "Enter the number of processes: ";
    cin >> num_processes;

    vector<Process> processes(num_processes);
    cout << "\nEnter PID, Arrival Time and Burst Time:\n";

    for (int i = 0; i < num_processes; i++) {
        cin >> processes[i].pid
            >> processes[i].arrival_time
            >> processes[i].burst_time;
    }

    double fcfs_tat;
    double fcfs_wt = FCFS(processes, fcfs_tat);

    // Run SJF
    double sjf_tat;
    double sjf_wt = SJF(processes, sjf_tat);

    cout << "\n===== COMPARISON =====\n";

    cout << fixed << setprecision(2);

```

```

cout << "\n===== COMPARISON =====\n";

cout << fixed << setprecision(2);

cout << "FCFS Average WT : " << fcfs_wt << endl;
cout << "SJF Average WT : " << sjf_wt << endl;

cout << "FCFS Average TAT : " << fcfs_tat << endl;
cout << "SJF Average TAT : " << sjf_tat << endl;

// SJF better if BOTH WT and TAT are lower
if (sjf_wt < fcfs_wt && sjf_tat < fcfs_tat) {

    cout << "\nSJF has better performance than FCFS." << endl;

}
else if (fcfs_wt < sjf_wt && fcfs_tat < sjf_tat) {

    cout << "\nFCFS has better performance than SJF." << endl;

}
else {

    if ((sjf_wt + sjf_tat) < (fcfs_wt + fcfs_tat)) {

        cout << "\nSJF has better performance than FCFS." << endl;

    }
    else if ((fcfs_wt + fcfs_tat) < (sjf_wt + sjf_tat)) {

        cout << "\nFCFS has better performance than SJF." << endl;

    }
    else {

        cout << "\nBoth FCFS and SJF have similar performance." << endl;

    }

}

return 0;
}

```

Figure 2: Source Code of the Comparison of FCFS vs SJF Algorithm

4. Output Screenshot:

```
Enter the number of processes: 5
Enter PID, Arrival Time, Burst Time for each process:
p1 3 3
p2 2 5
p3 5 4
p4 1 3
p5 6 2

PID AT  BT  CT  TAT WT
p1  3   3   7   4   1
p2  2   5  18  16  11
p3  5   4  13   8   4
p4  1   3   4   3   0
p5  6   2   9   3   1

Average Waiting Time: 3.40
Average Turnaround Time: 6.80
```

Figure 3: Execution Output of the SJF Algorithm


```

Enter the number of processes: 5

Enter PID, Arrival Time and Burst Time:
P1 3 3
P2 2 5
P3 5 4
P4 1 3
P5 6 2

===== FCFS =====
PID    AT    BT    CT    TAT    WT
P4      1     3     4     3     0
P2      2     5     9     7     2
P1      3     3    12     9     6
P3      5     4    16    11     7
P5      6     2    18    12    10

Average Waiting Time: 5.00
Average Turnaround Time: 8.40

===== SJF =====
PID    AT    BT    CT    TAT    WT
P1      3     3     7     4     1
P2      2     5    18    16    11
P3      5     4    13     8     4
P4      1     3     4     3     0
P5      6     2     9     3     1

Average Waiting Time: 3.40
Average Turnaround Time: 6.80

===== COMPARISON =====
FCFS Average WT : 5.00
SJF Average WT  : 3.40
FCFS Average TAT : 8.40
SJF Average TAT  : 6.80

SJF has better performance than FCFS.

...Program finished with exit code 0
Press ENTER to exit console.

```

Figure 4: Execution Output of the Comparison of FCFS vs SJF Algorithm

5. Discussion:

The implementation of the **Shortest Job First–Non-Preemptive (SJF-NP)** scheduling algorithm provides an optimal solution for minimizing the average waiting time among non-preemptive scheduling algorithms, assuming that the burst times are known in advance. The algorithm continuously selects the process with the shortest burst time from the pool of processes that have already arrived and are ready for execution.

The implementation uses a while loop that continues until all processes are completed. In each iteration, it searches through all processes to identify the incomplete process with the minimum burst time among those that have already arrived. If no process is available at the current time, indicating a **CPU idle situation**, the algorithm advances the current time to the arrival time of the next process.

The important characteristics observed in this implementation are:

- The algorithm provides the minimum possible average waiting time among non-preemptive scheduling algorithms when burst times are known.
- It handles **CPU idle time** by advancing the current time to the next process arrival time when no process is ready.
- The selection process requires **$O(n)$** time for each scheduling decision.
- **Starvation** may occur if shorter processes continuously arrive before longer processes get an opportunity to execute.
- The algorithm generally provides better average waiting-time performance than **FCFS**.

Overall, the implementation effectively demonstrates the trade-off between **optimal scheduling performance and computational complexity**. Although SJF-NP can minimize average waiting time, it requires prior knowledge or accurate estimation of burst times, which may not always be practical in real-world operating systems.

6. Conclusion:

The **Shortest Job First–Non-Preemptive (SJF-NP)** scheduling algorithm successfully minimizes the average waiting time when the burst times of processes are known in advance. It is particularly effective for **batch processing systems** where the workload is predictable and accurate burst-time information is available. However, predicting the actual burst time of processes can be difficult in real-world systems. Furthermore, SJF may cause **starvation** for longer processes if shorter processes continue to arrive.

Despite these limitations, SJF is an important CPU scheduling algorithm because it provides a theoretical benchmark for evaluating the performance of other scheduling algorithms and demonstrates the principle of minimizing average waiting time.

First-Come, First-Served (FCFS) is required because it is a simple, fair, and easy-to-implement scheduling algorithm. It schedules processes according to their arrival order and does not require prior knowledge of burst times. Therefore, FCFS is useful when the execution time of processes is unknown or difficult to predict. It also provides a basic reference point for comparing more advanced scheduling algorithms such as SJF.

However, FCFS can suffer from the **convoy effect**, where a long process delays several shorter processes, resulting in a higher average waiting time.

SJF generally performs better than FCFS when there are **multiple processes with significantly different burst times**, especially when several short processes are available. By executing shorter processes first, SJF can significantly reduce the average waiting time compared with FCFS.

For example, if a long process arrives before several short processes, FCFS may force all the short processes to wait until the long process finishes. SJF, on the other hand, can execute the shorter processes first, resulting in a lower average waiting time.

Therefore, **FCFS is preferred for simplicity and fairness**, while **SJF is preferred when minimizing average waiting time is the primary objective and burst times are known or can be estimated accurately**.